University of Jeddah, KSA
College of Computer Science and Engineering

Final Project Report

Project Name: FIREGUARD IoT CAR

Team Members:

Full Name	ID
Toleen Wael	2311776
Yara Algamdi	2311765
Mead Al-Maabadi	2310895
Taif Eissa	2310874

Introduction

Fire accidents pose a serious risk to human life and property, especially in environments where immediate human intervention may be dangerous or delayed. With the advancement of robotics and Internet of Things (IoT) technologies, autonomous systems can be developed to assist in fire detection and suppression while minimizing human involvement. This project focuses on designing a smart fire-fighting robot that combines embedded systems, sensors, and cloud connectivity to improve safety and efficiency in fire response.

Project Objectives

- Detect fire using multiple flame sensors from different directions.
- Determine the direction of the fire source and navigate toward it autonomously.
- Control DC motors to enable forward and directional movement of the robot.
- Extinguish fire using an automated water pumping system.
- Control the water spray direction using a servo motor for accurate fire targeting.
- Integrate an ESP32 microcontroller to enhance system performance and connectivity.
- Enable IoT-based monitoring and control through wireless communication.
- Improve safety by minimizing human intervention in fire-extinguishing tasks.

Project Description

The Fire Fighter Robot is an IoT-based robotic system designed to detect, locate, and extinguish fire autonomously in small and controlled environments. The robot is equipped with multiple flame sensors positioned in different directions to continuously monitor its surroundings and accurately identify the direction of a fire source.

Once a flame is detected, the robot navigates toward the fire using DC motors controlled through a motor driver module. Upon reaching the fire location, a water pump is activated to suppress the fire, while a servo motor adjusts the direction of the water spray to ensure precise targeting.

The system is powered by an ESP32 microcontroller, which offers high processing performance and built-in Wi-Fi connectivity. Through a mobile dashboard, users can observe sensor data, control the water pump, and switch between automatic and manual operation modes.

This project demonstrates the effective use of robotics, embedded systems, and IoT technologies to enhance safety and reduce human risk in fire-fighting applications.

Hardware Components

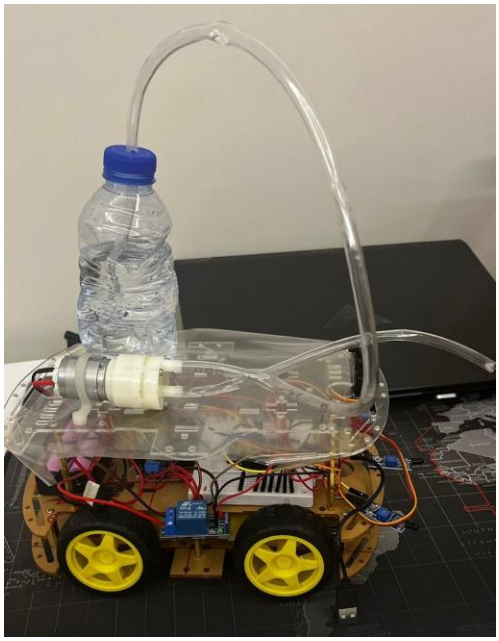
- ESP32 Microcontroller (Wi-Fi enabled)
- IR Flame Sensors × 3
- L298N Motor Driver Module
- BO DC Motors × 4
- Wheels × 4
- Mini Servo Motor (MG90S)
- 5–9V Water Pump
- Water Tank / Bottle
- Water Pipe (Tube)
- Breadboard (Solder-less)
- Jumper Wires (Male–Female)
- 3.7V Li-ion Batteries (18650) × 2
- Battery Holder (Series – 7.4V)
- TIP122 Transistor

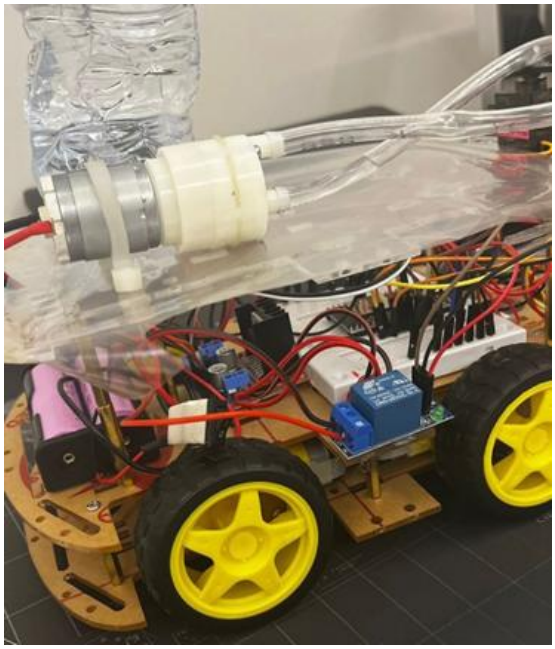
- IN4148 Diode
- 1K Ohm Resistor
- Capacitor (~100 pF)
- Robot Chassis

Software Components

- Arduino IDE
- ESP32 Board Package
- Embedded C/C++ Code
- IoT Cloud Platform
- Wi-Fi Network

System Architecture & Design





Implementation

The following ESP32 program controls the robot's movement, flame sensors, servo motor, and water pump operations.

```
#define min_val 25
#define enA 14
#define in1 26
#define in2 27
#define in3 33
#define in4 25
#define enB 15
#define ir_R 34
#define ir_F 35
#define ir_L 32
#define servo 18
#define pump 19
const int Speed=160;
const int pwmChannelA=0;
const int pwmChannelB=1;
const int pwmFreq=1000;
const int pwmResBits=8;
int s1,s2,s3;
void pump_on(){pinMode(pump,OUTPUT);digitalWrite(pump,LOW);}
void pump_off(){pinMode(pump,INPUT_PULLUP);digitalWrite(pump,HIGH);}
void
setup(){Serial.begin(9600);pinMode(ir_R,INPUT);pinMode(ir_F,INPUT);pinMode(ir_
L,INPUT);pinMode(in1,OUTPUT);pinMode(in2,OUTPUT);pinMode(in3,OUTPUT);pin
Mode(in4,OUTPUT);pinMode(servo,OUTPUT);pump_off();ledcSetup(pwmChannelA,
pwmFreq,pwmResBits);ledcSetup(pwmChannelB,pwmFreq,pwmResBits);ledcAttac
hPin(enA,pwmChannelA);ledcAttachPin(enB,pwmChannelB);for(int
angle=90;angle<=140;angle+=5){servoPulse(servo,angle);}for(int
angle=140;angle>=40;angle-=5){servoPulse(servo,angle);}for(int
angle=40;angle<=95;angle+=5){servoPulse(servo,angle);}ledcWrite(pwmChannelA,
Speed);ledcWrite(pwmChannelB,Speed);delay(500);}
void
loop(){s1=analogRead(ir_R);s2=analogRead(ir_F);s3=analogRead(ir_L);Serial.print(
s1);Serial.print("\t");Serial.print(s2);Serial.print("\t");Serial.println(s3);delay(50);if
(s1<min_val){Stop();pump_on();for(int angle=90;angle>=40;angle-
=3){servoPulse(servo,angle);}for(int
angle=40;angle<=90;angle+=3){servoPulse(servo,angle);}}else
if(s2<min_val){Stop();pump_on();for(int
angle=90;angle<=140;angle+=3){servoPulse(servo,angle);}for(int
```

```
angle=140;angle>=40;angle-=3){servoPulse(servo,angle);}for(int
angle=40;angle<=90;angle+=3){servoPulse(servo,angle);}}else
if(s3<min_val){Stop();pump_on();for(int
angle=90;angle<=140;angle+=3){servoPulse(servo,angle);}for(int
angle=140;angle>=90;angle-=3){servoPulse(servo,angle);}}else
if(s1>=min_val&&s1<=4000){pump_off();backword();delay(100);turnRight();delay(
200);}else if(s2>=min_val&&s2<=4000){pump_off();forword();}else
if(s3>=min_val&&s3<=4000){pump_off();backword();delay(100);turnLeft();delay(2
00);}else{pump_off();Stop();}delay(10);}
void servoPulse(int pin,int angle){int
pwm=(angle*11)+500;digitalWrite(pin,HIGH);delayMicroseconds(pwm);digitalWrit
e(pin,LOW);delay(50);}
void
forword(){digitalWrite(in2,HIGH);digitalWrite(in1,LOW);digitalWrite(in3,LOW);dig
italWrite(in4,HIGH);}
void
backword(){digitalWrite(in2,LOW);digitalWrite(in1,HIGH);digitalWrite(in3,HIGH);d
igitalWrite(in4,LOW);}
void
turnLeft(){digitalWrite(in2,LOW);digitalWrite(in1,HIGH);digitalWrite(in3,LOW);dig
italWrite(in4,HIGH);}
void
turnRight(){digitalWrite(in2,HIGH);digitalWrite(in1,LOW);digitalWrite(in3,HIGH);d
igitalWrite(in4,LOW);}
void
Stop(){digitalWrite(in1,LOW);digitalWrite(in2,LOW);digitalWrite(in3,LOW);digital
Write(in4,LOW);}
```

Tools and Technologies Used

The following tools were used in the development and testing of the Fire Fighter Robot project:

- **Arduino IDE** – for writing and uploading the code to the ESP32.
- **ESP32 Development Board** – as the main controller with built-in Wi-Fi.
- **Blynk Cloud and Mobile Application** – for remote monitoring and control.
- **Embedded C/C++** – for implementing the system logic.
- **Laptop and USB Cable** – for programming and testing.

System Workflow

The system operates by continuously monitoring the environment using multiple flame sensors positioned on the robot. These sensors detect the presence and direction of fire and send the readings to the ESP32 microcontroller for processing. Based on the sensor data, the robot determines the appropriate movement direction and navigates toward the fire source using DC motors controlled by a motor driver module.

Once the robot reaches the fire location, the water pump is activated to extinguish the fire. At the same time, a servo motor adjusts the direction of the water spray to ensure accurate targeting of the flame. Throughout the operation, the ESP32 transmits sensor data and system status to the IoT cloud platform via Wi-Fi, allowing real-time monitoring and remote control. The system supports both automatic and manual modes.

Future Enhancements

In the future, the FireGuard IoT Car can be further enhanced to increase its efficiency, intelligence, and real-world applicability. Additional sensors such as smoke and gas sensors can be integrated to improve fire detection accuracy and enable the system to identify hazardous gases. Live camera monitoring can be added to allow remote visual supervision of the environment during operation. Moreover, developing a mobile application or web-based dashboard would enable users to monitor alerts, robot status, and fire locations in real time. Advanced autonomous path-planning techniques can also be implemented to help the robot navigate complex and dynamic environments more effectively. Finally, integrating GPS technology would allow real-time location tracking, making the system more suitable for large-scale environments such as forests and industrial zones.

Conclusion and System Enhancement

In this project, a Fire Fighter Robot was successfully designed and implemented using embedded systems and IoT technologies. Throughout the development process, design decisions were continuously evaluated to ensure that the system meets the project requirements efficiently. As part of this optimization, the initial microcontroller choice was revised to better align with the system's functionality.

The controller was upgraded from Arduino UNO to **ESP32** to support IoT-based features such as wireless communication and cloud integration. This enhancement allowed seamless connectivity with the cloud platform, enabling real-time monitoring and remote control of the robot. The final system demonstrates a reliable, efficient, and intelligent fire-fighting solution that integrates robotics, sensor technology, and cloud-based control, fulfilling the project objectives effectively.